



딥러닝 파이토치 8장: 성능 최적화

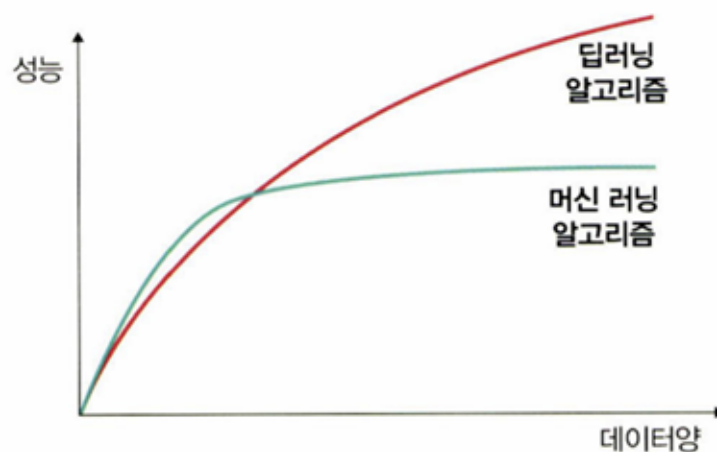
⚙ Status	Not started
👤 Assignee	👤 시현 이
📅 Due date	06/20/2026
🔧 Task type	개념맞코드필사
≡ Description	딥러닝 파이토치 8장: 성능 최적화

8.1 성능 최적화

8.1.1 데이터를 사용한 성능 최적화

- 多 데이터 수집

▼ 그림 8-1 데이터와 딥러닝, 머신 러닝 알고리즘의 성능 비교



- 데이터 생성

- 데이터 범위 조정
 - `시그모이드 활용 -> 데이터셋 범위 0~1
 - tanh 활용 → -1~1의 값

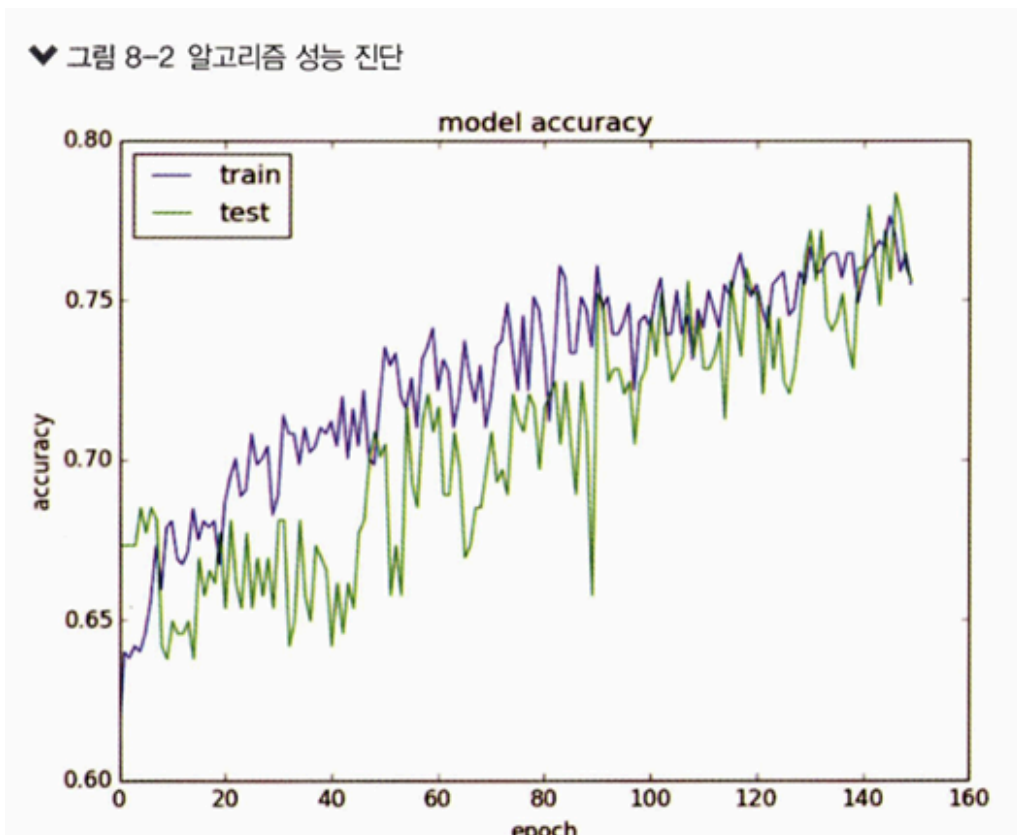
8.1.2 알고리즘을 이용한 성능 최적화

유사한 용도의 알고리즘들을 선택해 모델을 훈련시켜보고 최적의 성능을 보이는 알고리즘 선택

8.1.3 알고리즘 튜닝을 위한 성능 최적화

가장 많은 시간이 소요되는 부분. 하이퍼파라미터 조정

- 진단: 모델에 대한 평가를 바탕으로 모델이 overfitting인지 혹은 다음 원인으로 성능 향상에 문제가 있는건지에 대한 insight 얻기
 - ex) 훈련 성능이 검증보다 눈에 띄게 좋음 → 규제화 진행
 - ex) 훈련과 검증 결과가 모두 성능이 좋지 않음(과소적합) → epoch 수 조정
 - ex) 훈련 성능이 검증을 넘어서는 변곡점 존재 → 조기 종료 고려



- 가중치: 초깃값은 작은 난수를 사용. autoencoder 같은 비지도 학습을 이용해 사전 훈련 진행 후 지도학습 진행
- 학습률: 네트워크 구성에 따라 다름. 초기에 매우 크거나 작은 임의의 난수를 선택
 - 네트워크 계층이 多 → 학습률 높이기
 - 네트워크 계층이 少 → 학습률 작게
- 활성화 함수: 활성화 함수 변경 시, 손실함수도 함께 변경
- 배치와 에포크: 트렌드-큰 에포크와 작은 배치 but, 적절한 배치 크기글 위해 훈련데이터셋의 크기와 동일 or 하나의 배치로 훈련시켜보는 등 다양한 테스트 진행
- 옵티마이저 및 손실함수:
 - 일반적으로 확률적 경사 하강법을 많이 사용
 - Adam이나 RMSProp 등도 사용
- 네트워크 구성(network topology)
 - 여러 성능 테스트 → 하나의 은닉층에 뉴런 여러개(네트워크 넓음), 네트워크 계층 증가와 뉴런 개수 감소(네트워크 깊음) or 두개 다 결합

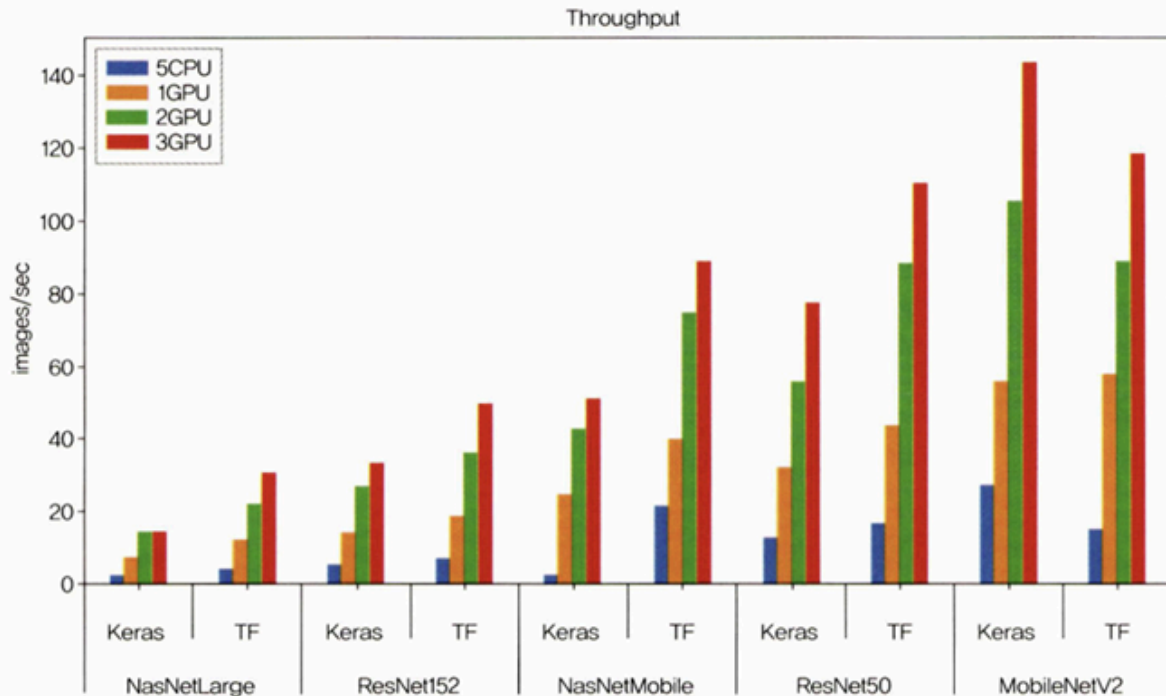
8.1.4 앙상블을 이용한 성능 최적화

앙상블: 모델 두 개 이상 혼합

8.2 하드웨어를 이용한 성능 최적화

8.2.1 CPU와 GPU 사용의 차이

▼ 그림 8-3 CPU와 GPU 성능 비교¹⁾



→ CPU 5개를 동시에 돌려도 GPU 한 개보다 성능이 좋지 못함

- CPU : 연산을 담당하는 ALU와 명령어를 해석하고 실행하는 컨트롤, 그리고 데이터를 담아두는 캐시로 구성돼 있음
 - 명령어가 입력되는 순서대로 데이터를 처리하는 직렬 처리 방식
 - 한번에 하나의 명령어만 처리하기 때문에 연산을 담당하는 ALU 개수가 많은 필요가 없음
- GPU : 병렬 처리를 위해 개발됨
 - 캐시 메모리 비중은 낮고, 연산을 수행하는 ALU 개수 많아짐
 - 서로 다른 명령어를 동시에 병렬적으로 처리하도록 설계됨
 - 하나의 코어에 ALU 수백~수천 개가 장착되어 있기 때문에 CPU로는 시간이 많이 걸리는 3D 그래픽 작업 등을 빠르게 수행 가능
- 개별적 코어 속도: CPU > GPU
- 사용 환경 비교

- python이나 matlab 처럼 행렬 연산을 많이 사용하는 재귀 연산이 대표적인 '직렬' 연산을 수행(CPU 유리)
- 역전파처럼 복잡한 미적분은 병렬 연산(GPU 적합)

8.2.2 GPU를 이용한 성능 최적화

- CUDA: NVIDIA에서 개발한 GPU 개발 툴

8.3 하이퍼파라미터를 이용한 성능 최적화

8.3.1 배치 정규화를 이용한 성능 최적화

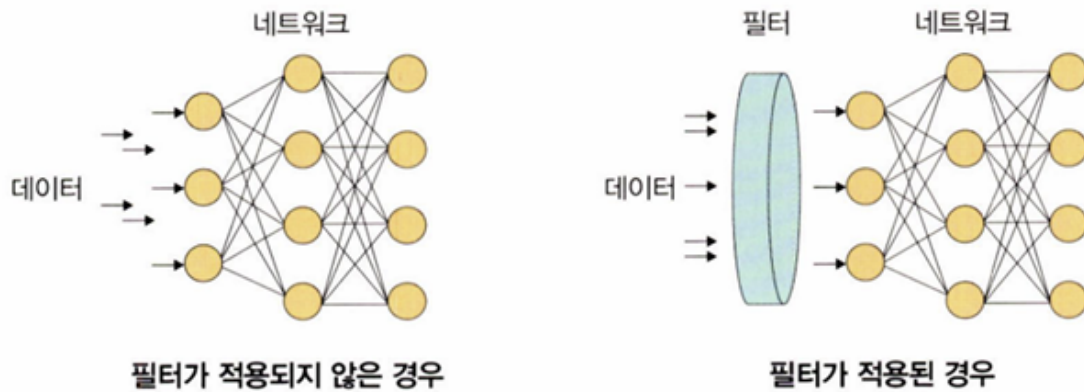
- 정규화
 - 데이터 범위를 사용자가 원하는 범위로 제한하는 것
 - 각 특성 범위scale을 조정한다는 의미로 특성 스케일링이라고도 함
 - 스케일 조정을 위해 MinMaxScaler()기법을 사용

$$\frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

(x: 입력 데이터)

- 규제화
 - 모델 복잡도를 줄이기 위해 제약을 두는 방법(필터 적용과 비슷)
 - 드롭아웃, 조기 종료 등의 방법

▼ 그림 8-32 규제화



- 표준화
 - 기존 데이터를 평균 0, 표준편차는 1인 형태의 데이터로 만드는 방법
 - standard scaler 혹은 z-score normalization이라고 함
 - 평균을 기준으로 얼마나 떨어져 있는지 살펴볼 때 사용

$$\frac{x - m}{\sigma}$$

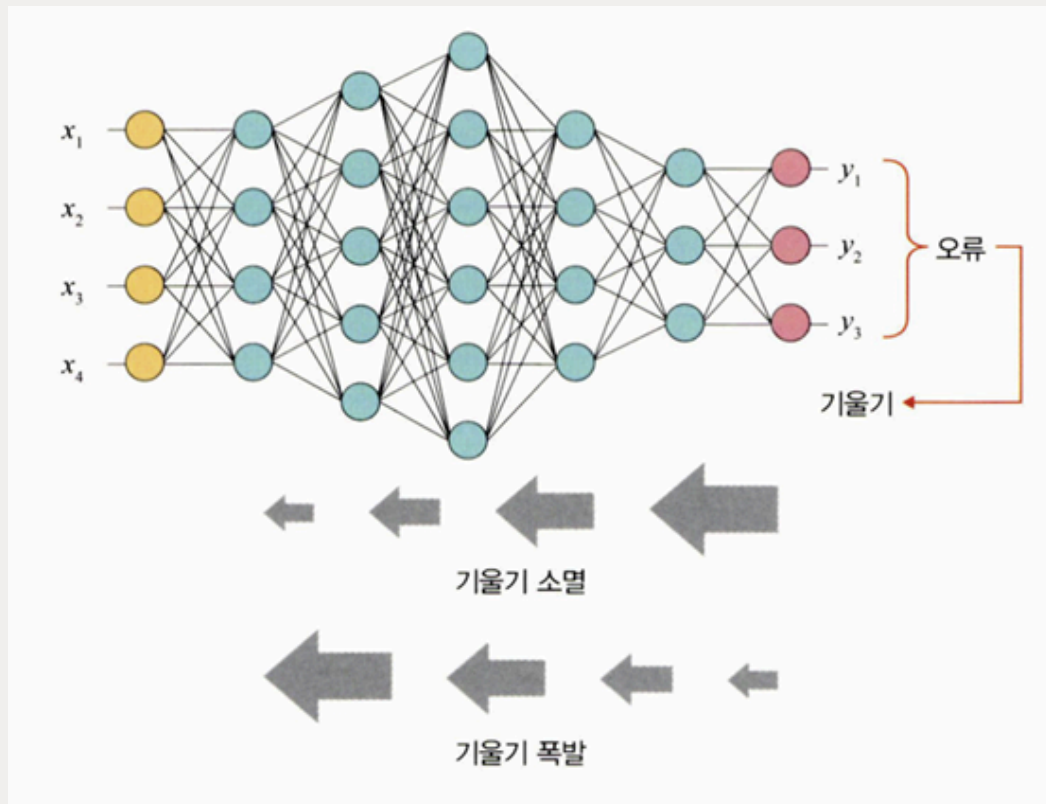
(σ : 표준편차, x : 관측 값(입력 값), m : 평균)

- 배치 정규화
 - "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift" 논문 기법 → 데이터 분포가 안정되어 학습 속도 높일 수 있음
 - 기울기 소멸이나 기울기 폭발 같은 문제 해결

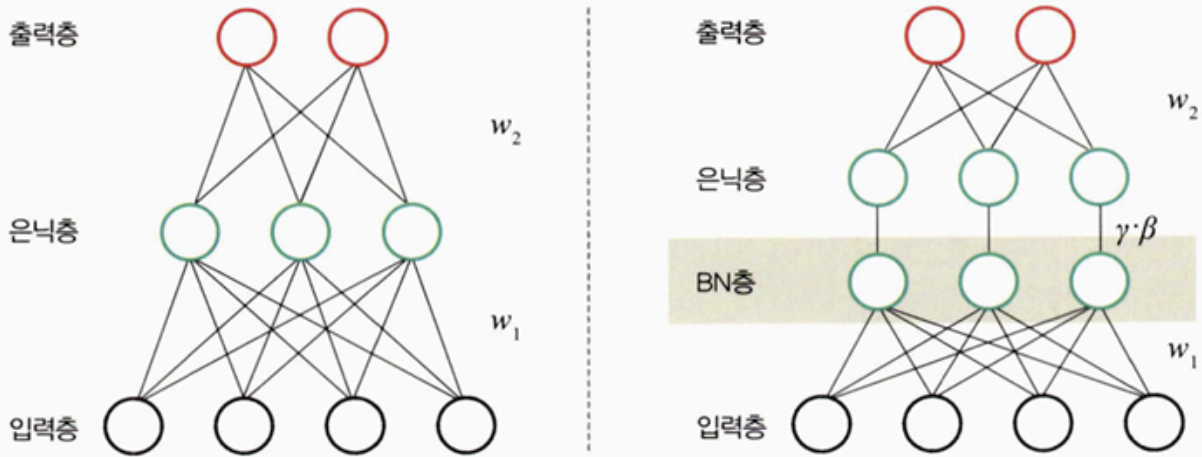


기울기 소멸과 기울기 폭발

- 기울기 소멸: 오차 정보를 역전파시키는 과정에서 기울기가 급격히 0에 가까워져 학습이 되지 않는 현상
- 기울기 폭발: 학습 과정에서 기울기가 급격히 커지는 현상



▼ 그림 8-35 배치 정규화



- 기울기 소멸과 폭발의 원인은 internal covariance shift 때문
 - 네트워크의 각 층마다 활성화 함수가 적용되면서 입력 값들의 분포가 계속 바뀌는 현상
 - 분산된 분포를 정규분포로 만들기 위해 표준화와 유사한 방식을 mini-batch에 적용해 평균은 0으로, 표준편차는 1로 유지하도록 함

$$\mu\beta \leftarrow \frac{1}{m} \sum_{i=1}^m x_i \quad \text{----①}$$

$$\sigma^2\beta \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu\beta)^2 \quad \text{----②}$$

$$\hat{x}_i \leftarrow \frac{x_i - \mu\beta}{\sqrt{\sigma^2\beta + \epsilon}} \quad \text{----③}$$

$$y_i \leftarrow \gamma \hat{x}_i + \beta \Leftrightarrow BN_{\gamma, \beta}(x_i) \quad \text{----④}$$

$$\left(\begin{array}{l} \text{입력: } \beta = \{x_1, x_2, \dots, x_n\} \\ \text{학습해야 할 하이퍼파라미터: } \gamma, \beta \\ \text{출력: } y_i = BN_{\gamma, \beta}(x_i) \end{array} \right)$$

1. 미니 배치 평균

2. 미니 배치의 분산과 표준편차

3. 정규화 수행

4. scale을 조정

⇒매 단계마다 활성화 함수를 거치며 데이터셋 분포가 일정

But, 단점 존재

1. 배치 크기가 작을 때는 정규화 값이 기존 값과 다른 방향으로 훈련될 수 있음

a. 분산이 0이면 정규화 자체가 안됨

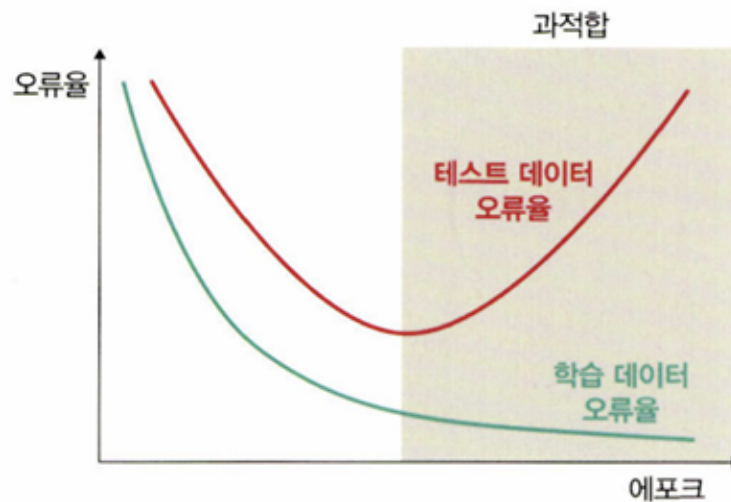
2. RNN은 네트워크 계층별로 미니 정규화를 적용해야하기 때문에 모델이 더 복잡해지면서 비효율적일 수 있음

→ 해결하기 위해 가중치 수정, 네트워크 구성 변경 등 수행 BUT, 궁극적으로 배치 정규화로 성능이 좋아지기에 많이 사용됨

8.3.2 드롭아웃을 이용한 성능 최적화

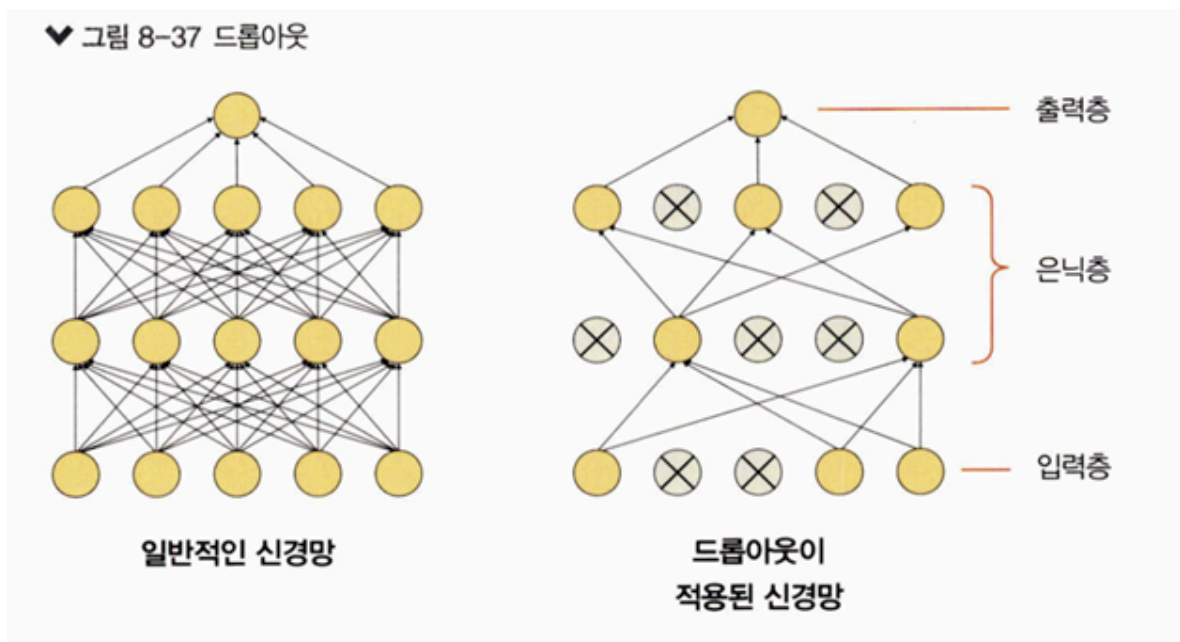
과적합: 훈련 데이터셋을 과하게 학습하는 것

▼ 그림 8-36 훈련과 오류율



- 드롭아웃:

- 훈련할 때 일정 비율의 뉴런만 사용하고, 나머지 뉴런에 해당하는 가중치는 업데이트하지 않는 방법
- 매 단계마다 사용하지 않는 뉴런을 바꾸어 가며 훈련시킴]
- 노드를 임의로 끄면서 학습하는 방법으로, 은닉층에 배치된 노드 중 일부를 임의로 끄면서 학습. 꺼진 노드는 신호를 전달x → 지나친 학습 방지
- 테스트 데이터로 평가할 때는 노드를 모두 사용해 출력하되 드롭아웃 비율을 곱해서 성능 평가
- 훈련시간이 길어지는 단점, BUT 성능 좋아짐



8.3.3 조기 종료를 이용한 성능 최적화

- early stopping은 뉴럴 네트워크가 과적합을 회피하는 규제 기법
- 훈련 데이터와 별도로 검증 데이터 준비하고 매 epoch마다 검증 데이터에 대한 오차를 측정하여 모델의 종료 시점을 제어
 - 과적합 발생 시, 훈련 데이터셋에 대한 오차는 감소, 검증 데이터셋에 대한 오차는 증가
 - 오차가 증가하는 시점에서 학습 stop

▼ 그림 8-50 조기 종료

